Computación y estructuras discretas II
Grupo 3 Semestre 2026-1

SOLUCIÓN — Seguimiento #3 **A**

Nombre: _____ Código: _____

P.1. **(2.5 pts)** Usando el **método del árbol de recursión**, analice y resuelva la siguiente relación de recurrencia:

$$T(n) \;=\; 4\,T\left(\tfrac{n}{2}\right) + (n+3), \qquad T(1) = 1$$

Solución.

Parámetros: $a = 4, \quad b = 2, \quad f(n) = n + 3.$

Expansión de subproblemas:

$$T\left(\tfrac{n}{2}\right) = 4T\left(\tfrac{n}{4}\right) + \left(\tfrac{n}{2} + 3\right) \qquad T\left(\tfrac{n}{4}\right) = 4T\left(\tfrac{n}{8}\right) + \left(\tfrac{n}{4} + 3\right)$$

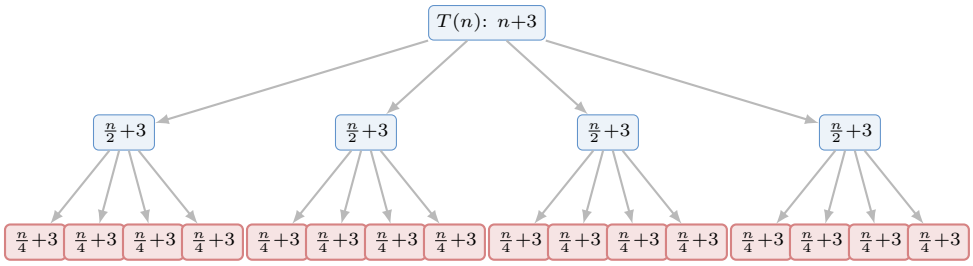

Tabla de análisis:

Nivel i	Nódos	Tamaño subprob.	Costo por nodo	Costo total del nivel
0	1	n	$n+3$	$n+3$
1	4	$n/2$	$\frac{n}{2}+3$	$4\cdot\left(\frac{n}{2}+3\right) = 2n+12$
2	$4^2 = 16$	$n/4$	$\frac{n}{4}+3$	$16\cdot\left(\frac{n}{4}+3\right) = 4n+48$
i	4^i	$n/2^i$	$\frac{n}{2^i}+3$	$4^i\cdot\left(\frac{n}{2^i}+3\right) = n\cdot\frac{4^i}{2^i} + 3\cdot 4^i = n\cdot 2^i + 3\cdot 4^i$

Altura del árbol. El caso base ocurre cuando $n/2^i = 1$, es decir $2^i = n$:

$$h = \log_2 n$$

Número de hojas: $4^h = 4^{\log_2 n} = n^{\log_2 4} = n^2$.
Costo total de las hojas: $n^2 \cdot T(1) = n^2$.

Sumatoria de los nodos internos (niveles $i = 0$ hasta $\log_2 n - 1$):

$$S = \sum_{i=0}^{\log_2 n - 1} \left(n \cdot 2^i + 3 \cdot 4^i\right) = n \sum_{i=0}^{\log_2 n - 1} 2^i \; + \; 3 \sum_{i=0}^{\log_2 n - 1} 4^i$$

Aplicando la fórmula de serie geométrica $\displaystyle\sum_{k=0}^{m-1} r^k = \frac{r^m - 1}{r - 1}$:

$$\sum_{i=0}^{\log_2 n - 1} 2^i = \frac{2^{\log_2 n} - 1}{2 - 1} = n - 1$$

$$\sum_{i=0}^{\log_2 n - 1} 4^i = \frac{4^{\log_2 n} - 1}{4 - 1} = \frac{n^{\log_2 4} - 1}{3} = \frac{n^2 - 1}{3}$$

Por lo tanto:

$$S = n(n-1) + 3 \cdot \frac{n^2 - 1}{3} = n(n-1) + (n^2 - 1) = n^2 - n + n^2 - 1 = 2n^2 - n - 1$$

Polinomio total (nodos internos + costo hojas):

$$T(n) = S + n^2 = (2n^2 - n - 1) + n^2 = \boxed{3n^2 - n - 1}$$

El término de mayor grado es $3n^2$, luego:

$$\mathcal{O}(n^2)$$

a	b	d	Caso del Teorema Maestro	Complejidad
4	2	1	$a > b^d \;\Rightarrow\; 4 > 2^1 = 4 > 2$ (Caso 3)	$\mathcal{O}(n^{\log_2 4}) = \mathcal{O}(n^2)$ ✓

Verificación: Ambos métodos producen $\mathcal{O}(n^2)$. Coincide con árbol de recursión: **Sí** ✓

P.2. **(2.5 pts)** Usando reconocimiento de patrones en una función recursiva de cola, programe en **Scala** usando listas recursivas una función que reciba como parámetro una lista de enteros positivos y verifique si la lista es **estrictamente creciente**.

```
def esEstrictamenteCreciente(l: List[Int]): Boolean
```

Solución.
La función auxiliar interna `aux` recibe la lista restante y el último elemento visto. Los casos base (lista vacía y un solo elemento) retornan `true` directamente. En el caso recursivo, se compara `head` con `prev`: si `head <= prev` la condición se rompe y retorna `false`; de lo contrario continúa con `head` como nuevo `prev`.

```
import scala.annotation.tailrec

@tailrec
def esEstrictamenteCreciente(l: List[Int], prev: Int = 0): Boolean =
  l match {
    case Nil           => true
    case head :: tail => if (head <= prev) false else esEstrictamenteCreciente(tail, head
        )
  }
```

Verificación de los ejemplos:

Ejemplo 1: `List(1, 3, 5, 8, 12, 15)`

head	prev	head > prev	**Acción**
3	1	Sí	continúa
5	3	Sí	continúa
8	5	Sí	continúa
12	8	Sí	continúa
15	12	Sí	continúa
`Nil`	–	–	retorna `true` ✓

Ejemplo 2: `List(2, 4, 4, 7, 9)`

head	prev	head > prev	**Acción**
4	2	Sí	continúa
4	4	No	retorna `false` ✓